Analysis and Optimisation of CloverLeaf on a Kunpeng 920 Node

Performance Engineering, Group 11

Jort Oosterveld Sergiu-Adrian Preda

Faculty of Science and Engineering, University of Groningen

October 2026

Baseline

The application and the machine

CloverLeaf (reference, MPI + OpenMP)

- Fluid dynamics simulator
- Large vectors, many nested loops
- Forces computed for each cell every timestep
- Compute-heavy *kernels*, both FORTRAN and C implementations
- We focus on C only, FORTRAN as reference
- benchmark **bm3840**: 3840^2 cells, 87 steps, ≈25 arrays of 118 MB ≈3 GB working set

HAICGU node

- 2× Kunpeng 920 (ARMv8, 2.6 GHz): 128 cores, 4 NUMA domains
- 16 channels DDR4-2666, 128 GB, L3 2×64 MB
- GCC 14.1, OpenMPI 5.0, PAPI 7.1, Linux `perf`
- `-O3 -march=native -funroll-loops -fopenmp`
- layout "$r \times t$": r ranks × t threads, one thread per core, ranks pinned

Research question: how far is the baseline from what the hardware allows, and what does it take to close the gap?

Methodology

1. CloverLeaf's own profiler: wall clock per phase
2. PAPI per kernel, one set per core/thread: IPC, instruction mix, stalls
3. Linux `perf` and the DDR-controller PMU: **bytes that cross the memory interface**, read around every run

Protocol. 5 repetitions, median and min-max, every one under the DDR counters. All bandwidths are *hardware bytes* / wall clock, for both STREAM and Cloverleaf.

Checks of the tools

- Kernel times add up to the wall clock within 1 %
- DDR counters calibrated on STREAM: 32 B per count, no read-for-ownership on streaming stores
- NUMA page distribution is fair, busiest DDR controller group has a mean of 1.00

Correctness gate

`PASSED` means deviation of baseline run reproduced to 10^{-6} %.

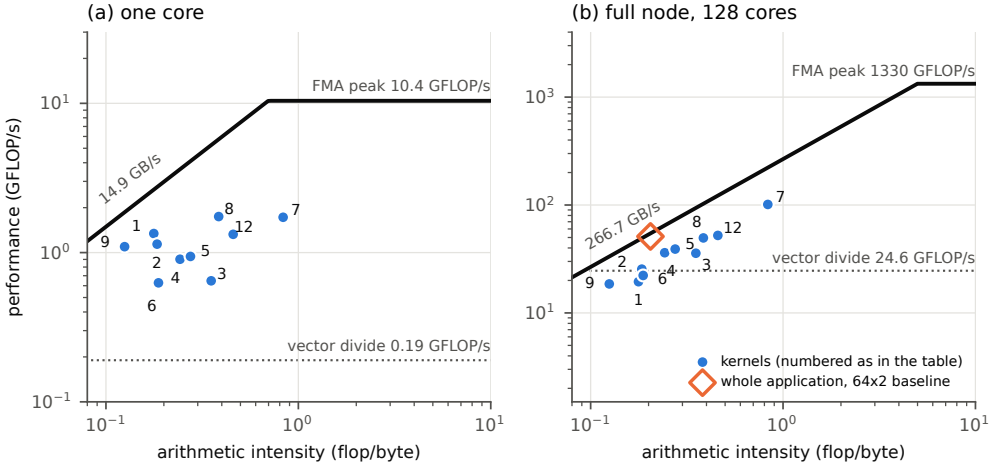
Ceilings, byte model and roofline

Measured ceilings

- STREAM triad, 128 threads: **266.5 GB/s**
- FMA peak **1330 GFLOP/s**
- Ridge at **5 flop/B**

Analytic byte model: arrays read and written per loop nest, write-allocate charged

Ceilings, byte model and roofline



university of groningen

Where the time goes

kernel	time share (%) 1 core	128 cores	IPC 64×2	stall (%) 64×2
`advec_mom`	28.6	**44.0**	0.27	91
`advec_cell`	19.4	**19.4**	0.28	91
`pdv` (2 calls)	17.6	9.6	0.21	92
`calc_dt`	11.8	4.8	0.50	81
`ideal_gas`	7.4	4.7	0.31	87
`viscosity`	5.7	2.2	0.99	62
`accelerate`	5.2	4.1	0.27	91
`reset` + `revert`	1.6	4.5	0.10	96
other (3)	2.7	6.7	–	–

bm3840, PAPI; 128 cores $= 1 \times 128$; stall = back-end stall cycles

- advection = 63 % of the node-level time; `advec_mom` scales worst (14× on 128 cores)
- at 64×2: IPC 0.1–0.5, **81–96 % back-end stalls**: the cores wait for memory
- `calc_dt`, `ideal_gas`, `viscosity` run *scalar*: `sqrt` stays a libm call without `-fno-math-errno`
- advection upwind loops: data-dependent indices, only 12–16 % vector instructions
- 12.5 % of all cycles in the OpenMP barrier (one per worksharing loop)

Baseline

kernels	layout	wall (s)	TB	% of roof
C	**64×2**	**11.32**	2.83	**94**
C	32×4	11.46	2.79	91
C	128×1	11.66	2.95	95
C	4×32	13.00	2.87	83
C	1×128	26.91	2.99	42
C	1×128, THP off	13.31	2.83	80
Fortran	64×2	9.33	**2.36**	95

On the bm3840 test set, roof = 266.5 GB/s

Finding

Memory performance already optimized, further gain has to come from **moving fewer bytes**, like the FORTRAN kernel.

- baseline 64×2: 250 GB/s over the whole run, start-up included: **94 % of the hardware ceiling**
- time floor for memory: $2.83\,\text{TB}/266.5\,GB/s = \mathbf{10.6\,s}$. Baseline is close to maximum memory performance.
- only 1×128 cares about THP: first touch puts whole arrays on one NUMA domain ($1.66\times$ imbalance)
- Fortran is 18 % faster, because it moves *fewer bytes*.

Kernel improvements

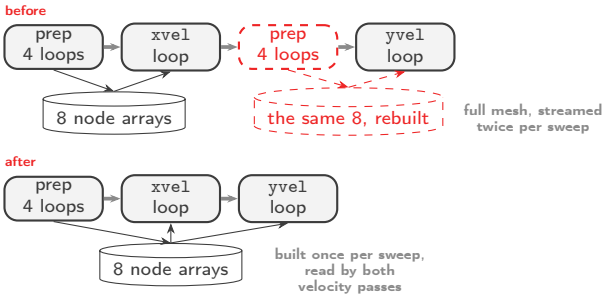
Plan: five byte reductions, predicted with the byte model

change	DRAM bytes	cumulative	floor (s)
(1) `advec_mom`: direction-only helper loops once per sweep, not once per velocity component	−12.8 %	−12.8 %	9.2
(2) `advec_cell`: four temporaries in registers instead of four full-mesh arrays	−5.9 %	−18.7 %	9.9
(3) dead stores `post_vol` / `pre_vol`: written over the haloed grid, never read	−4.4 %	−21.6 %	10.1
(4) Reset / Revert: swap the field allocations instead of copying 118 MB per field	−6.6 %	−28.2 %	9.8
(5) `calc_dt`: fuse the min reduction into the compute loop	−1.1 %	**−29.3 %**	**7.4**
(6) streaming stores (no read-for-ownership), upper bound	−22.8 %	−41.4 %	–

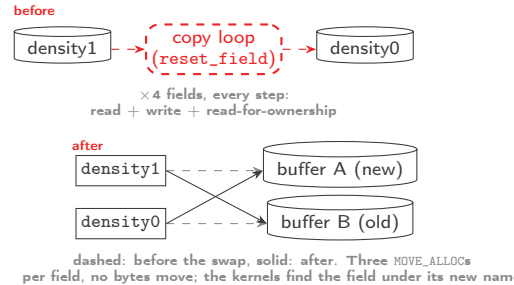
- selection: the three kernels with the largest gain in the model; (3) and (5) fell out while reading the code
- every change is an edit of the model's loop table, so prediction and measurement share one convention; time-only items (`nowait`, `-fno-math-errno`, branch-free upwind loop) come after the bytes

Data flow (1/2): work that was done twice

(1) `advec_mom`: helper loops once per sweep



(4) Reset / Revert: swap, not copy



Same for the `revert` kernel, this one is skipped as well.



university of groningen

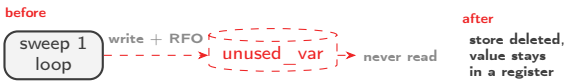
Data flow (2/2): round trips through DRAM

(2) `advec_cell`: temporaries in registers

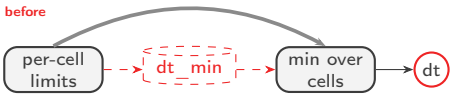


Same in both PdV passes: `volume_change`, stored and reloaded in one iteration, now a scalar.

(3) Dead stores (advec_cell & advec_mom)



(5) `calc_dt`: fused reduction



university of groningen

Results

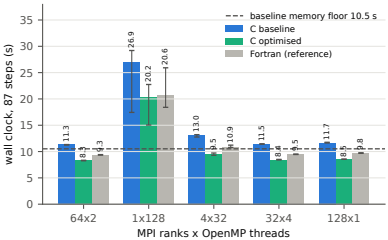
Ladder at 64×2: the wall clock follows the bytes

build	contents	wall (s)	TB	bytes	time
base	baseline source	11.31	2.83	–	–
flags	the two flags only	11.28	2.85	+0.7 %	−0.2 %
cell	flags + (2) (3) (4)	9.92	2.49	−12.1 %	−12.3 %
mom	flags + (1) (3) (5)	9.54	2.41	−14.9 %	−15.7 %
opt	all, with the flags	8.25	2.04	−27.8 %	−27.0 %
optnf	all, baseline flags	8.56	2.05	−27.4 %	−24.3 %
pdv	optnf + PdV scalar	**8.25**	**2.01**	**−28.8 %**	**−26.5 %**

jobs 34938, 35003 (optnf), 35016 (pdv): one node, 5 repetitions per build (spread $\leq 1\,\%$), DDR counters on

- the lines add up: $-28.8\,\%$ bytes against the $-30.8\,\%$ **the model predicted**
- every build sustains 240–252 GB/s: still on the roof, so time $\propto$ bytes
- the flags: 0 % on the baseline, 3.6 % on the optimised kernels, and they break the gate; **pdv** is the final build
- **1.37×** at 64×2: 11.32 s to 8.26 s

Every layout, every kernel



layout	64×2	32×4	128×1	4×32	1×128
speed-up	1.37	1.36	1.36	1.37	1.33

kernel	base (s)	pdv (s)	ratio
`advec_mom`	4.19	2.62	**1.60**
`advec_cell`	2.42	1.67	**1.45**
`pdv (both passes)`	1.32	1.17	1.13
`calc_dt`	0.66	0.62	1.05
`ideal_gas`	0.58	0.54	1.06
`reset + revert`	0.57	–	gone
`accel. + flux`	0.95	1.00	0.95

PAPI region times at 64×2, jobs 34798 and 35017 (pdv)

- the same quarter of the traffic gone at every layout, the same speed-up
- optimised C is now **faster than the Fortran reference at every layout**
- the application moved *up* the roof, not off it: 91 % of the ceiling, 9 % above the floor of its own bytes

university of groningen

Correctness, and what is still open

Gate on `clover_bm.in` (960^2, 2955 steps)

- the ladder's baseline binary reproduces the archived reference (0.0057 %) at every layout
- every build with the two compiler flags, flags-only included, gives *one and the same* other value (0.038 %): the flags alone move it (GCC's multiply-add contraction; bisect, job 34936)
- **optnf** (kernel changes, baseline flags) and **pdv** (optnf + PdV scalar) reproduce the reference at every layout (7 of 7 each)
- the flags were worth 3.6 % on top (vectorised `sqrt` in `calc_dt`, `ideal_gas`): a measured trade-off, decided for reproducibility

Limitations and next steps

- still on the bandwidth roof: 91 % of 266.5 GB/s, floor 7.6 s; `accelerate`, `flux_calc`, `viscosity` untouched: the next bytes to remove
- read-for-ownership of the C stores: up to -23 % more bytes with explicit streaming stores
- `nowait` and branch-free upwind loop still open in `advec_cell`: worth at most 9 %

Conclusions

1. The C kernels are memory-bound (0.2 flop/B against a 5 flop/B ridge) and the baseline already sustains 94 % of the 266.5 GB/s hardware ceiling: **only fewer bytes can make it faster**.
2. An analytic byte model, validated by the DDR counters to 1 %, predicted five changes at -29 % of the traffic, six at -31 %; measured: -28.8 %.
3. The wall clock followed the bytes: **1.37× at 64×2, 1.33–1.37× at every layout**; the optimised C build beats the Fortran reference everywhere and passes the gate.
4. The two compiler flags are worth 3.6 % on the optimised kernels but break bitwise reproducibility of the reference: dropped, as a measured trade-off. The next lever is the read-for-ownership of the stores, not instruction-side work.

Questions?